OBJECT-ORIENTED METHODOLOGIES (OOM)

Course Project Report

ArogyaNova

An Intelligent, Object-Oriented Prescription Management System

Submitted by

Shreya Kumbar — IIT205178

Anamika Chauhan — IIT2025151

Soumili Ghosh — IIT2025163

Academic Year 2026–27

Abstract

Paper prescriptions are easy to lose, hard to read and impossible to check automatically for dangerous drug combinations. Patients also forget doses, and doctors rarely learn whether a treatment was actually followed. **ArogyaNova** is a full-stack web application that digitises the complete prescription lifecycle: a doctor drafts a prescription, screens it for drug–drug interactions, finalises it, and the system then generates a medication schedule that the patient follows and logs, while pharmacists verify and administrators oversee the platform.

The project was built using **Object-Oriented Analysis and Design (OOAD)**. Requirements were captured as use cases, the domain was modelled as classes with UML, and the design was implemented in Python (FastAPI) and TypeScript (React) with a strict layered architecture. All four pillars of OOP — encapsulation, abstraction, inheritance and polymorphism — appear in real, tested business code, together with composition, generics, SOLID principles and design patterns such as Repository, Strategy, State, Observer-style notification and Dependency Injection.

Beyond core OOM, the project integrates **Database Management Systems** (a normalised 3NF relational schema with ACID transactions through an ORM), **Computer Networks** (a REST client–server design over HTTP/TCP with CORS), **Cybersecurity** (Argon2id hashing, JWT in HttpOnly cookies, role-based access control, rate limiting and security headers) and **AI** (a rule-based drug-interaction checker and a Tesseract OCR provider). It ships with a graphical user interface with light and dark themes and an automated test suite of 54 passing tests.

Keywords: *OOAD, UML, encapsulation, polymorphism, SOLID, design patterns, FastAPI, React, RBAC, drug-interaction checking.*

Table of Contents

Abstract 2
Table of Contents 3
1. Introduction 5
1.1 Background 5
1.2 Problem Statement 5
1.3 Objectives 5
1.4 Scope 5
1.5 Technology Stack 5
2. Requirement Analysis 7
2.1 Actors 7
2.2 Functional Requirements 7
2.3 Non-Functional Requirements 7
3. Object-Oriented Methodology 8
3.1 Phases Followed 8
3.2 Why an Object-Oriented Approach 8
4. System Architecture 9
5. UML Modelling 11
5.1 Use Case Diagram 11
5.2 Class Diagram — Domain Model 11
5.3 Class Diagram — Repository and Service Layer 12
5.4 Sequence Diagram 13
5.5 State Machine Diagram 13
5.6 Activity Diagram 14
6. OOP Concepts and Where They Are Used 15
6.1 Encapsulation 15
6.2 Abstraction 16
6.3 Inheritance 16
6.4 Polymorphism 17
6.5 Composition, Aggregation and Association 17
6.6 Generics, Enums and Data Classes 17
7. Design Principles and Design Patterns 19
7.1 SOLID Principles 19
7.2 Design Patterns 19
8. Advanced Concepts I — Database Management System 20
8.1 Normalisation 20
8.2 Integrity and Transactions 20

8.3 Sample Query (generated by the ORM) 21
9. Advanced Concepts II — Computer Networks 22
10. Advanced Concepts III — Cybersecurity 23
11. Advanced Concepts IV — Artificial Intelligence and Analytics 24
11.1 Drug–Drug Interaction Checker 24
11.2 Prescription OCR 24
11.3 Adherence Analytics 24
12. Graphical User Interface 25
13. Testing 30
14. Conclusion and Future Scope 31
14.1 Conclusion 31
14.2 Future Scope 31
References 31

(If page numbers are missing, right-click the table in Word and choose Update Field.)

1. Introduction

1.1 Background

A prescription is a legal, clinical and logistical document at the same time. In many clinics it is still handwritten, which leads to transcription errors, illegible dosages, lost records and no protection against harmful drug combinations. After the patient leaves, there is no reliable way to know whether the medicines were taken as directed.

1.2 Problem Statement

Design and implement a secure, role-aware system that manages the full prescription lifecycle — creation, safety checking, issuing, dispensing review, scheduled intake and adherence tracking — using object-oriented methodologies, so that the business rules of a prescription are enforced by the software itself rather than by user discipline.

1.3 Objectives

- Model the clinical domain (users, prescriptions, medications, schedules) with well-encapsulated classes.
- Use UML (use case, class, sequence, state, activity, ER and deployment diagrams) to analyse and document the system.
- Demonstrate every core OOP concept in working code, and apply SOLID principles and recognised design patterns.
- Provide a usable graphical interface for four roles: Doctor, Patient, Pharmacist and Administrator.
- Integrate DBMS, computer-networking, cybersecurity and AI concepts into one coherent product.
- Verify behaviour with automated unit tests.

1.4 Scope

The system covers authentication, user and role management, a medication catalogue, prescription authoring with line items, a drug-interaction safety check, prescription OCR, automatic schedule generation, dose logging, notifications and analytics. Billing, insurance and live integration with pharmacy inventory are out of scope.

1.5 Technology Stack

Table 1: Technology stack

Layer	Technology	Why it was chosen
GUI (frontend)	React 19, TypeScript, Vite, Tailwind CSS, Recharts	Component-based, typed, fast development server, charts for analytics
API (backend)	Python 3.11, FastAPI, Uvicorn (ASGI)	Asynchronous, type-hinted, automatic OpenAPI docs

Layer	Technology	Why it was chosen
Validation	Pydantic v2 schemas	Declarative data contracts between layers
Persistence	SQLAlchemy 2.0 (async ORM), SQLite / PostgreSQL, Alembic	Object-relational mapping, portable SQL, schema migrations
Security	argon2-cffi, python-jose (JWT), SlowAPI	Modern password hashing, stateless auth tokens, rate limiting
AI	Rule-based interaction engine, pytesseract OCR	Explainable clinical checks, offline text extraction
Testing	pytest, pytest-asyncio	Unit tests for domain, repositories and RBAC

2. Requirement Analysis

2.1 Actors

Table 2: System actors

Actor	Description	Key goals
Doctor	Licensed prescriber	Create, check, finalise and cancel prescriptions; view patients
Patient	Receiver of care	View prescriptions, follow schedule, log doses
Pharmacist	Dispensing professional	Review and verify prescriptions, screen interactions
Administrator	System owner	Manage users and roles, view analytics and audit logs

2.2 Functional Requirements

Table 3: Functional requirements

ID	Requirement
FR-1	Users register and log in; the system issues a signed session token.
FR-2	Each role sees only the screens and data it is permitted to access.
FR-3	A doctor creates a draft prescription for a patient with one or more medication items.
FR-4	Items can be added or removed only while the prescription is a draft.
FR-5	The doctor can run an AI drug-interaction check on the selected medicines.
FR-6	Finalising a prescription requires at least one item and generates a medication schedule.
FR-7	Prescriptions can be cancelled with a reason; completed or expired ones cannot change.
FR-8	Patients log each scheduled dose as taken or skipped; adherence is computed.
FR-9	Users receive in-app notifications about prescription events.
FR-10	Administrators manage users and view analytics; reports can be exported as CSV.

2.3 Non-Functional Requirements

- **Security:** passwords never stored in plain text; every API call authenticated and authorised.
- **Maintainability:** layered architecture, single-responsibility classes, automated tests.
- **Portability:** runs with zero configuration on SQLite, or on PostgreSQL via Docker.
- **Usability:** responsive GUI with light and dark themes and one-click demo logins.
- **Performance:** asynchronous I/O end to end; paginated list endpoints.

3. Object-Oriented Methodology

Object-oriented methodology views a system as a collection of collaborating objects, each combining data with the behaviour that operates on that data. The project followed the **Object-Oriented Analysis and Design (OOAD)** process, which unifies ideas from Booch, Rumbaugh's Object Modelling Technique (OMT) and Jacobson's use-case driven Objectory approach — the same ideas later merged into UML and the Unified Process.

3.1 Phases Followed

Table 4: OOAD phases

Phase	Activity in this project	Artefact produced
Inception / Analysis	Identified actors and use cases from the clinical workflow	Use case diagram, requirement tables
Domain modelling	Found nouns (Prescription, Medication, Doctor…) and their responsibilities	Class diagram, ER diagram
Dynamic modelling	Traced how objects collaborate per use case; modelled lifecycle states	Sequence, state and activity diagrams
Design	Assigned layers, applied SOLID and patterns, defined interfaces	Architecture & repository diagrams
Implementation	Coded domain classes first, then services, repositories, API and GUI	Python and TypeScript source
Testing & deployment	Unit tests for domain rules and RBAC; Docker deployment	54 passing tests, deployment diagram

3.2 Why an Object-Oriented Approach

A prescription has rules that must always hold — for example, an issued prescription must never be silently edited. In a procedural design these checks are scattered across every screen that touches the data. In an object-oriented design the rule lives inside the `Prescription` class, which is the only thing allowed to change its own status. This is the central reason the project uses OOM: correctness is guaranteed by the structure of the code, not by remembering to call a validation function.

4. System Architecture

ArogyaNova uses a **clean layered architecture**. Each layer depends only on the layer beneath it, and the domain layer depends on nothing external. This makes the business rules testable without a database or web server.

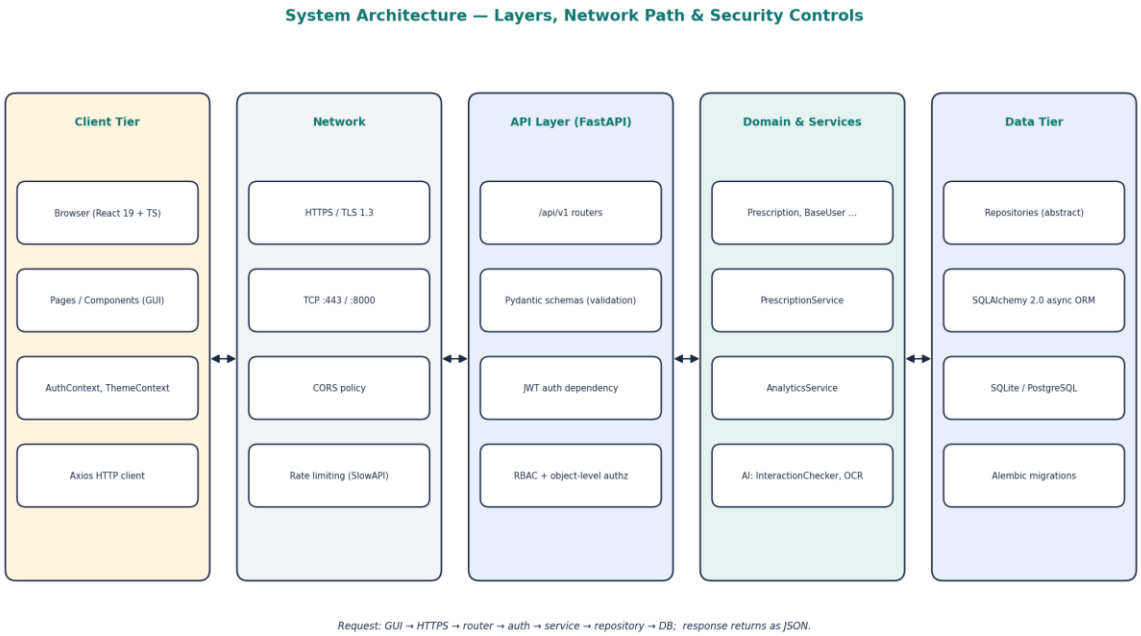


Figure 1: Layered architecture and request path

Table 5: Layers and their responsibilities

Layer	Folder	Responsibility
Presentation (GUI)	frontend/src	Pages, components, contexts (auth, theme), HTTP service classes
API / Controller	backend/app/api/v1	HTTP routing, request parsing, response formatting
Schemas	backend/app/schemas	Pydantic data-transfer objects and validation
Services	backend/app/services	Application workflows (finalise, cancel, analytics)
Domain	backend/app/domain	Pure business objects and rules: users, prescription, AI, notifications
Repositories	backend/app/repositories	Abstract data-access contracts and SQLAlchemy implementations
Persistence	backend/app/models, db	ORM models, database engine, seed data
Security	backend/app/security	Password hashing, JWT handling, RBAC dependencies

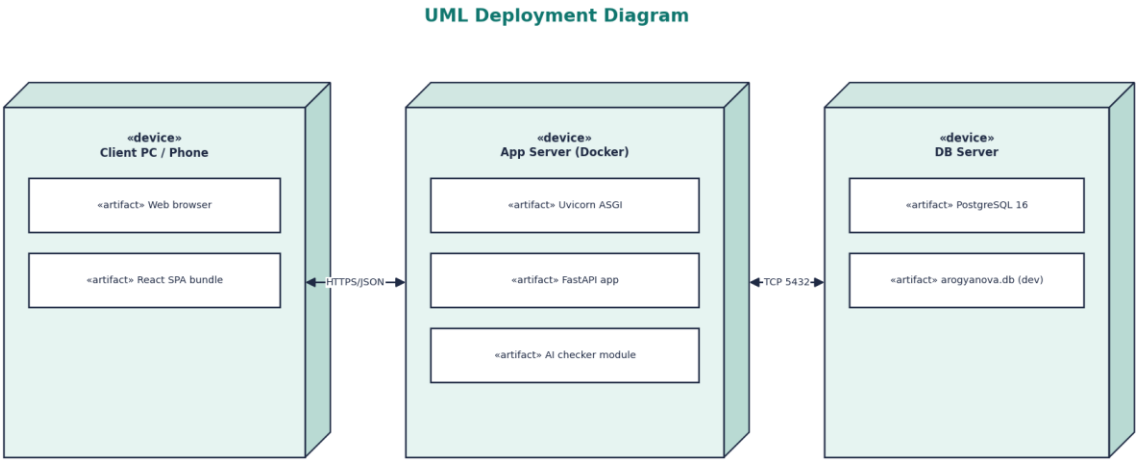


Figure 2: UML deployment diagram

5. UML Modelling

5.1 Use Case Diagram

The use case diagram captures the functional view from the four actors' perspective. «include» shows that finalising always sends a notification; «extend» shows that the AI interaction check optionally extends finalisation.

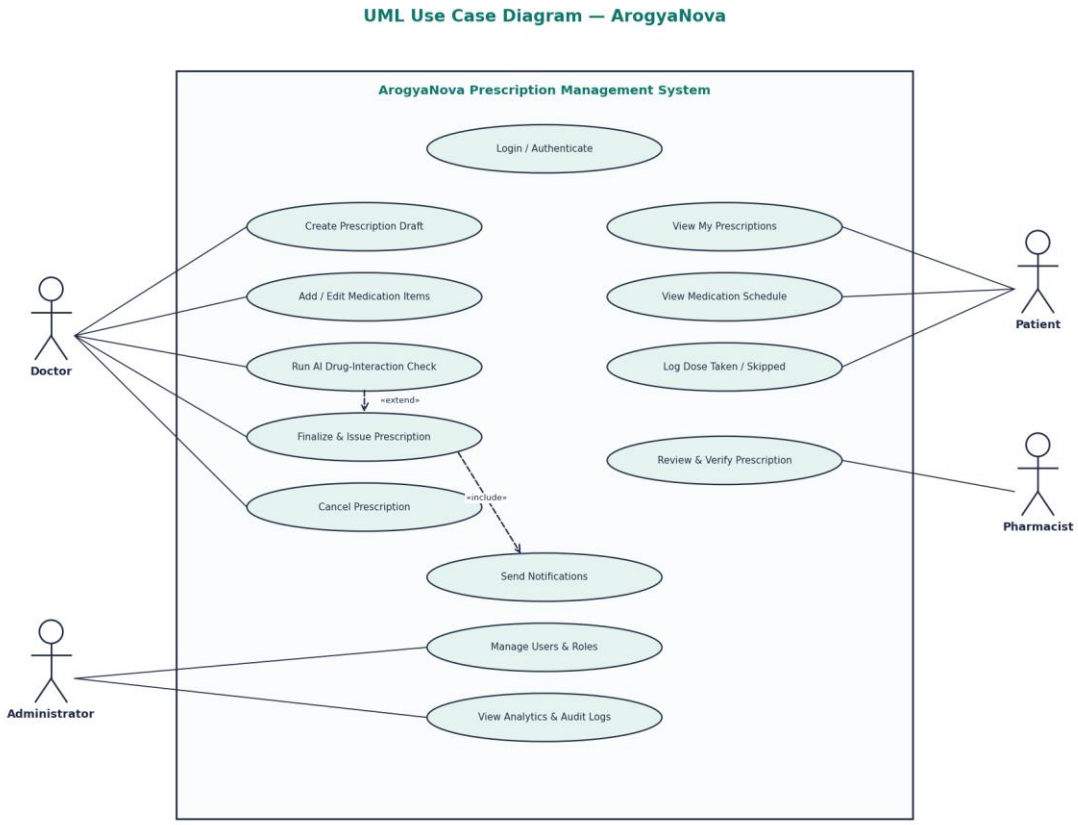


Figure 3: Use case diagram

5.2 Class Diagram — Domain Model

The domain class diagram is the heart of the design. `BaseUser` is an abstract class specialised by four roles (generalisation). `Prescription` owns its `PrescriptionItem` objects (composition — items cannot exist without their prescription), and each item references a `Medication` from the catalogue (association). The AI and notification capabilities are expressed as interfaces with several interchangeable implementations.

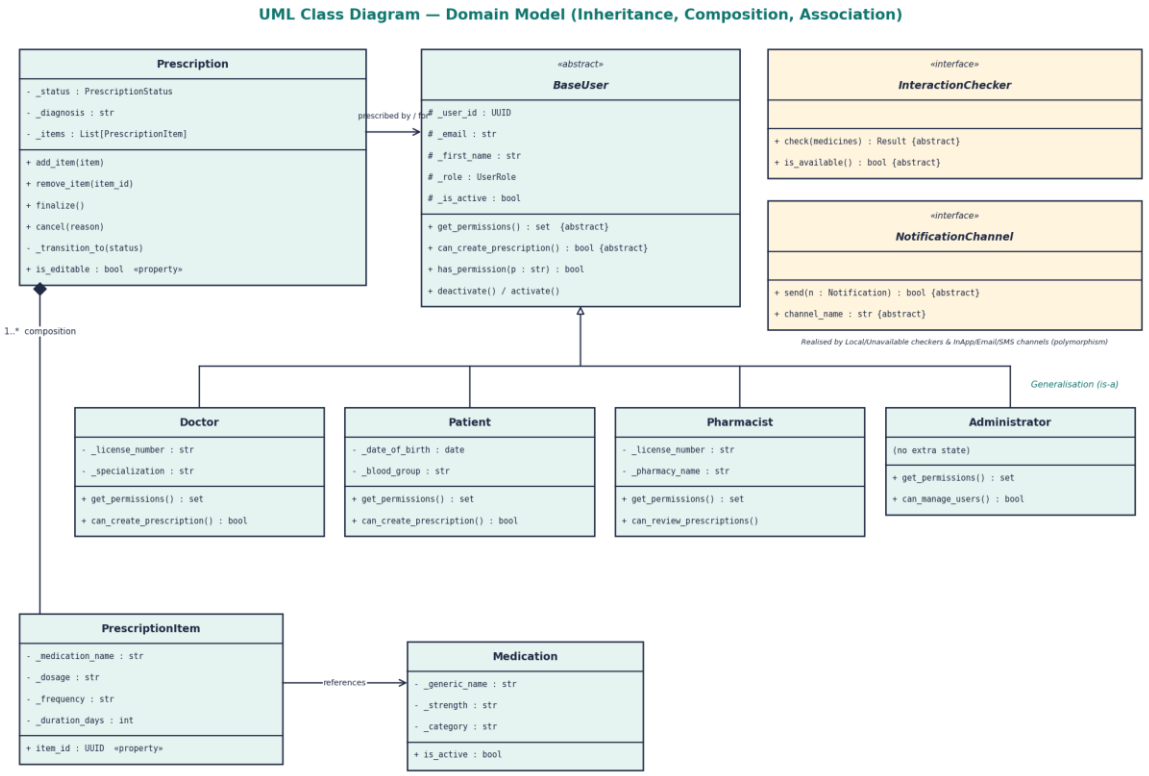


Figure 4: Domain class diagram

5.3 Class Diagram — Repository and Service Layer

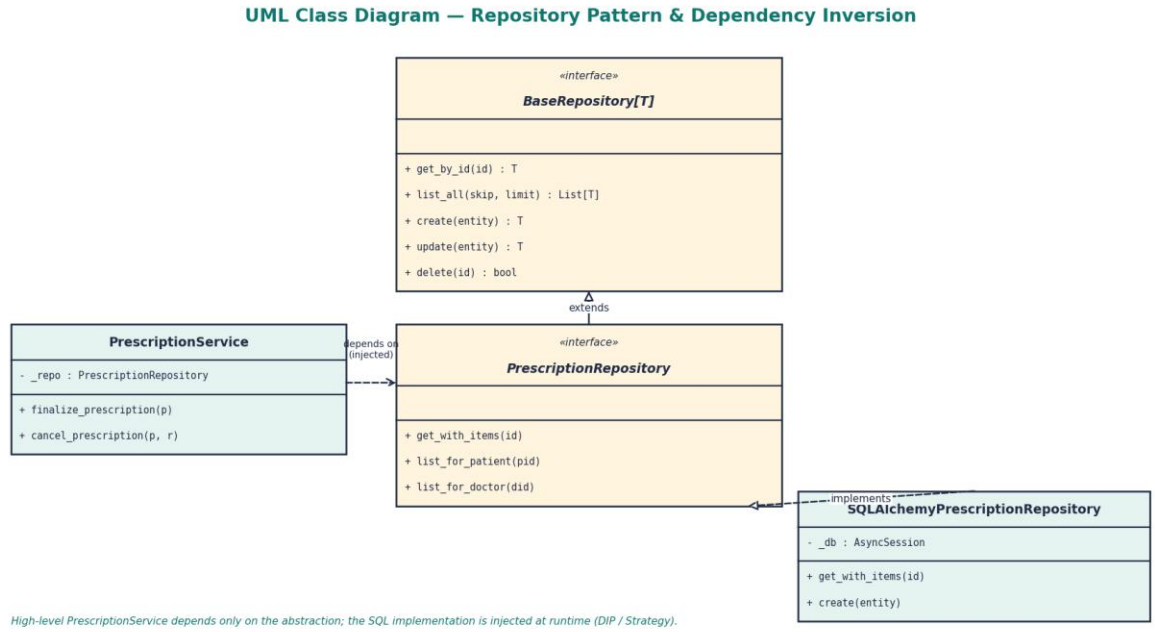


Figure 5: Repository pattern and dependency inversion

5.4 Sequence Diagram

The sequence diagram shows the message flow when a doctor finalises a prescription — from the GUI, across the network to the API, through authentication, the AI checker, the service, the repository and the database, and finally to the notification service.

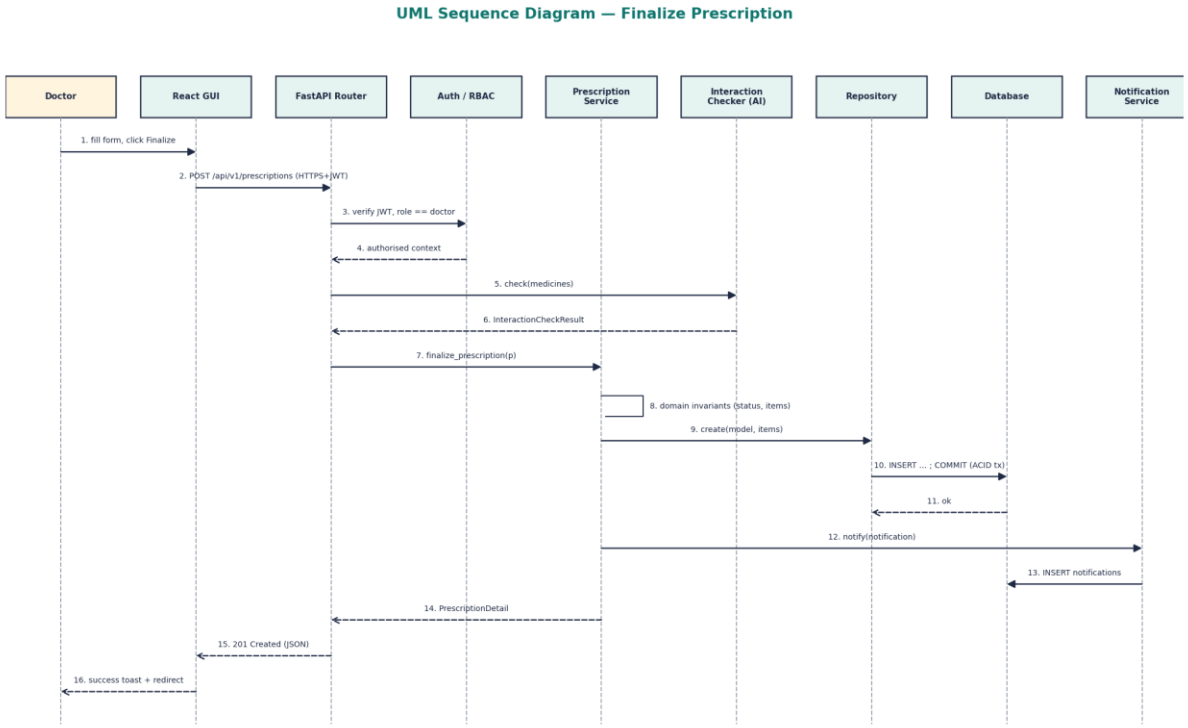


Figure 6: Sequence diagram: finalise prescription

5.5 State Machine Diagram

A prescription moves through five states. The allowed transitions are stored in a class-level table, `_VALID_TRANSITIONS`, and checked by the private method `_transition_to()`. Any illegal move raises an error, so the diagram is enforced by code rather than just documented.

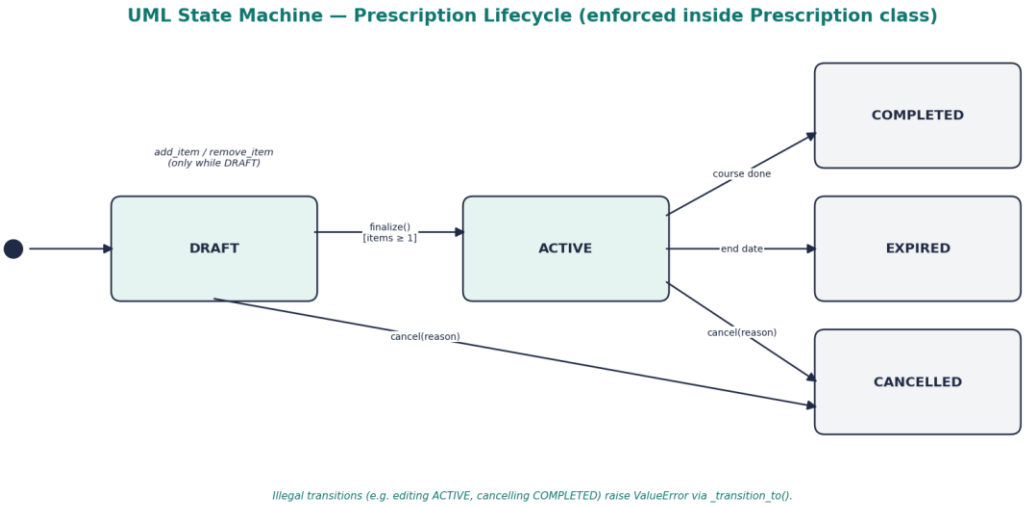


Figure 7: Prescription state machine

5.6 Activity Diagram

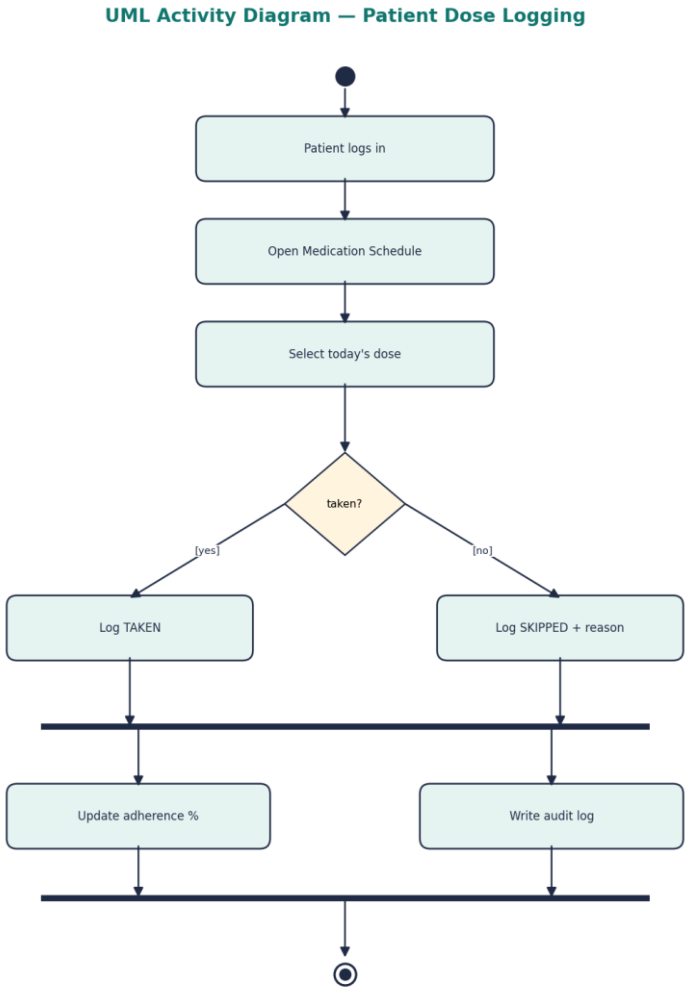


Figure 8: Activity diagram: patient logs a dose

6. OOP Concepts and Where They Are Used

This chapter maps each object-oriented concept to the exact class and file where it is implemented. All file paths are relative to `arogyanova/backend/app` unless stated.

Table 6: OOP concept map

Concept	Where in the code	How it is used
Class & Object	domain/prescription.py, domain/users.py	Prescription, PrescriptionItem, Doctor, Patient… are instantiated per record
Encapsulation	Prescription, BaseUser	Private _ attributes, read-only @property accessors, controlled mutators
Abstraction	BaseUser, BaseRepository, InteractionChecker, OCRProvider, NotificationChannel	ABC classes with @abstractmethod define what, not how
Inheritance	Doctor / Patient / Pharmacist / Administrator ← BaseUser	Shared identity fields reused through super().__init__()
Polymorphism (overriding)	get_permissions(), can_create_prescription(), send(), check()	Same call, different behaviour per subclass at runtime
Composition	Prescription ◆— PrescriptionItem; NotificationService ◆— channels	Whole owns parts; lifetime bound to the owner
Association	Prescription → Patient, Doctor; Item → Medication	Objects reference each other by identity
Generics	BaseRepository(ABC, Generic[T])	One typed CRUD contract reused for every entity
Enumerations	UserRole, PrescriptionStatus, RouteOfAdministration, InteractionSeverity	Type-safe closed sets of values
Data classes	Notification (frozen), DrugInteraction, OCRResult	Immutable value objects with auto-generated methods
Magic methods	__repr__, __post_init__	Readable debugging output and post-construction validation
Exception handling	_transition_to(), add_item(), FastAPI handlers	Invalid operations raise errors translated to HTTP 4xx
Interfaces (TypeScript)	frontend/src/types/index.ts	Typed contracts for API data on the GUI side

6.1 Encapsulation

Encapsulation bundles data with the methods that protect it. In `Prescription`, every attribute is private by convention (leading underscore). Outside code can **read** the status through a property but cannot **assign** it; the only way to change state is through intention-revealing methods that enforce the rules. The `items` property even returns a defensive copy so callers cannot modify the internal list.

```
class Prescription:
    @property
    def status(self) -> PrescriptionStatus:
        return self._status                    # read-only

    @property
    def items(self) -> list[PrescriptionItem]:
        return list(self._items)               # defensive copy

    def add_item(self, item: PrescriptionItem) -> None:
        if not self.is_editable:
            raise ValueError(f"Cannot add items to a prescription in '{self._status}' status
.")
        self._items.append(item)
        self._touch()

    def finalize(self) -> None:
        if not self._items:
            raise ValueError("A prescription must have at least one medication before it can
 be finalized.")
        self._transition_to(PrescriptionStatus.ACTIVE)
```

6.2 Abstraction

Abstraction hides implementation detail behind a contract. `BaseUser` is an abstract base class: it cannot be instantiated, and every concrete role must implement `get_permissions()` and `can_create_prescription()`. The same idea appears in `BaseRepository[T]` (data access), `InteractionChecker` and `OCRProvider` (AI) and `NotificationChannel` (messaging).

```
class BaseUser(ABC):
    @abstractmethod
    def get_permissions(self) -> set[str]: ...

    @abstractmethod
    def can_create_prescription(self) -> bool: ...

    def has_permission(self, permission: str) -> bool:   # concrete, shared
        return permission in self.get_permissions()
```

6.3 Inheritance

Inheritance models an **is-a** relationship. A Doctor *is a* user, so `Doctor` extends `BaseUser`, reuses its constructor through `super().__init__()`, and adds only what is specific to doctors — a licence number, a specialisation and a permission set. `Patient`, `Pharmacist` and `Administrator` follow the same pattern, which removes duplicated identity code.

```
class Doctor(BaseUser):
    PERMISSIONS = {"prescription:create", "prescription:finalize",
                   "prescription:cancel", "ai:interaction_check", ...}

    def __init__(self, user_id, email, first_name, last_name,
                 license_number, specialization=None, ...):
        super().__init__(user_id, email, first_name, last_name, UserRole.DOCTOR, ...)
        self._license_number = license_number
        self._specialization = specialization
```

6.4 Polymorphism

Polymorphism lets one interface have many forms. **Runtime (subtype) polymorphism** is used throughout: the calling code holds a reference typed as the abstract class and calls a method, and the correct override runs depending on the actual object.

- `user.has_permission("prescription:finalize")` returns True for a `Doctor` and False for a `Patient`, because each overrides `get_permissions()`.
- `NotificationService.notify()` loops over a list of `NotificationChannel` objects and calls `send()` on each; in-app, email and SMS channels each deliver differently.
- `InteractionChecker.check()` is served by `LocalInteractionChecker` normally, or by `UnavailableInteractionChecker` when the engine is offline — the caller never changes.

```
class NotificationService:
    def __init__(self, channels: list[NotificationChannel]) -> None:
        self._channels = channels                    # composition + DIP

    async def notify(self, notification: Notification) -> dict[str, bool]:
        results = {}
        for channel in self._channels:
            if channel.is_available():
                results[channel.channel_name] = await channel.send(notification)  # polymorp
hic call
        return results
```

Compile-time style polymorphism appears as optional/default parameters (e.g. `cancel(reason=None)`) and in TypeScript through union types and generic API helpers in the frontend.

6.5 Composition, Aggregation and Association

- **Composition:** a `Prescription` owns a private list of `PrescriptionItem` objects. Items are only added through the prescription and are deleted with it (cascade delete in the database).
- **Aggregation:** `NotificationService` holds a collection of channels that exist independently and are passed in from outside.
- **Association:** a prescription refers to a patient and a doctor; an item refers to a catalogue `Medication`.
- **Dependency:** `PrescriptionService` uses a `PrescriptionRepository` received through its constructor.

6.6 Generics, Enums and Data Classes

```
T = TypeVar("T")

class BaseRepository(abc.ABC, Generic[T]):
    @abc.abstractmethod
    async def get_by_id(self, entity_id: uuid.UUID) -> Optional[T]: ...
    @abc.abstractmethod
    async def create(self, entity: T) -> T: ...

class PrescriptionStatus(str, Enum):
    DRAFT = "draft"; ACTIVE = "active"; COMPLETED = "completed"
    CANCELLED = "cancelled"; EXPIRED = "expired"
```

```
@dataclass(frozen=True)
class Notification:            # immutable value object
    notification_id: uuid.UUID
    user_id: uuid.UUID
    title: str
    message: str
    priority: NotificationPriority = NotificationPriority.NORMAL
```

7. Design Principles and Design Patterns

7.1 SOLID Principles

Table 7: SOLID principles

Principle	Application in ArogyaNova
Single Responsibility	Prescription only guards its own state; repositories only run queries; services orchestrate; routers only handle HTTP.
Open / Closed	A new channel (e.g. WhatsApp) or a new AI checker is added by writing a new subclass — existing service code is unchanged.
Liskov Substitution	Any BaseUser subclass, InteractionChecker or NotificationChannel can replace its parent without breaking callers.
Interface Segregation	Small, focused interfaces: OCRProvider is separate from InteractionChecker; PrescriptionRepository adds only prescription queries.
Dependency Inversion	PrescriptionService and NotificationService depend on abstractions that are injected, not on concrete database or SMTP classes.

7.2 Design Patterns

Table 8: Design patterns used

Pattern	Type	Where	Benefit
Repository	Architectural	repositories/base.py and implementations	Business code is independent of SQL and the database engine
Strategy	Behavioural	InteractionChecker, OCRProvider implementations	Algorithm swapped at runtime (local vs unavailable)
State (table-driven)	Behavioural	Prescription._VALID_TRANSITIONS	Lifecycle rules centralised and enforced
Observer / Publish–subscribe	Behavioural	NotificationService fan-out to channels	Many subscribers notified of one event
Null Object	Behavioural	UnavailableInteractionChecker, UnavailableOCRProvider	Safe, honest fallback instead of crashes or fake results
Dependency Injection	Creational	FastAPI Depends(), service constructors	Loose coupling; easy test doubles
Template Method	Behavioural	BaseUser.has_permission() calls abstract get_permissions()	Shared algorithm, customised step
Data Transfer Object	Structural	Pydantic schemas	Validated boundary between API and domain
Context / Provider (GUI)	Structural	AuthContext, ThemeContext (React)	Shared state without prop drilling

8. Advanced Concepts I — Database Management System

Persistent data is stored in a relational database accessed through **SQLAlchemy 2.0**, an object-relational mapper (ORM). The ORM maps each table to a Python class, bridging the object world of the domain with the relational world of SQL. The same code runs on **SQLite** (zero-configuration development) and **PostgreSQL** (production / Docker).

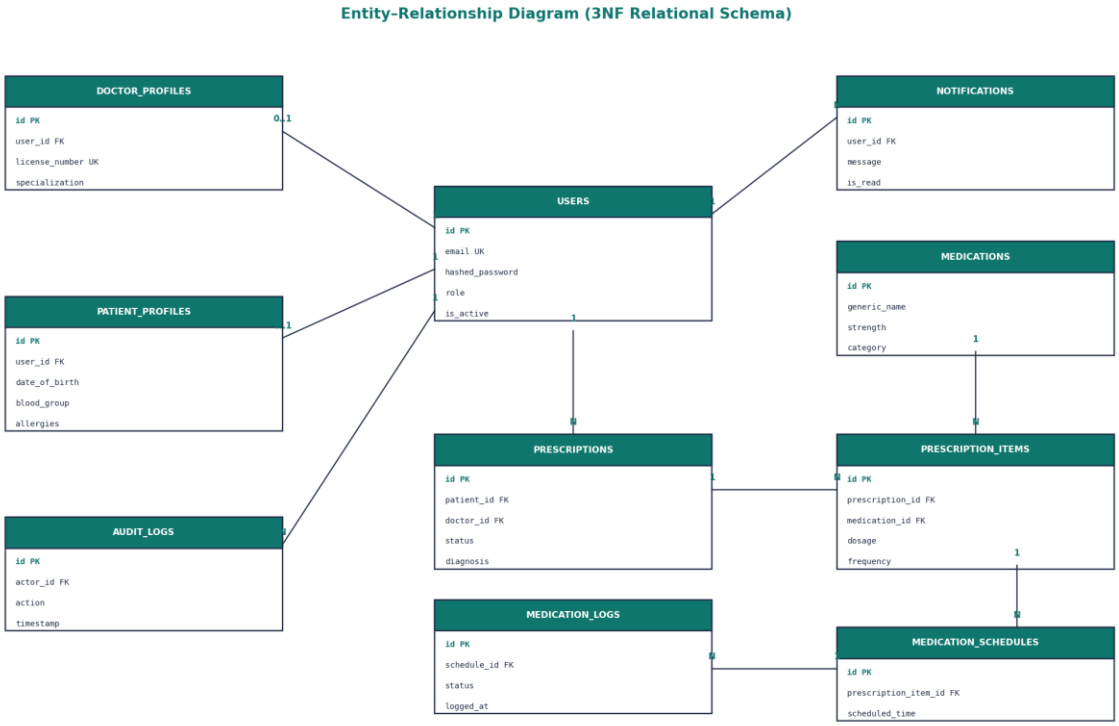


Figure 9: Entity–relationship diagram

8.1 Normalisation

- **1NF:** every column holds a single atomic value; medicines of a prescription are rows in PRESCRIPTION_ITEMS, not a comma-separated list.
- **2NF:** every table has a single-column surrogate primary key (UUID), so no partial dependencies exist.
- **3NF:** role-specific data lives in separate profile tables (DOCTOR_PROFILES, PATIENT_PROFILES…), and medication details live once in MEDICATIONS, removing transitive dependencies.

8.2 Integrity and Transactions

- Primary keys (UUID), foreign keys and unique constraints (e.g. email, licence number) enforce entity and referential integrity.
- Finalising a prescription inserts items and schedule rows inside one transaction — either everything is committed or nothing is (**atomicity**), keeping the database **consistent**.

- Asynchronous sessions with a connection pool give **isolation** between concurrent requests; committed data is **durable** on disk.
- Eager loading (`selectinload`) avoids the N+1 query problem when lists of users or prescriptions are returned.
- Alembic migrations version the schema like source code.

8.3 Sample Query (generated by the ORM)

```
SELECT users.* FROM users
WHERE users.role = 'patient' AND users.is_active = 1
ORDER BY users.last_name
LIMIT 20 OFFSET 0;

SELECT patient_profiles.* FROM patient_profiles
WHERE patient_profiles.user_id IN (...);   -- selectinload: one extra query, not N
```

9. Advanced Concepts II — Computer Networks

ArogyaNova is a **client–server** system. The React GUI runs in the browser (client) and talks to the FastAPI server over the network using a **RESTful** interface.

Table 9: Networking concepts by layer

OSI / TCP-IP layer	How it appears in the project
Application	HTTP/1.1 methods (GET, POST, PATCH, DELETE), JSON bodies, status codes 200/201/401/403/404/422/429/500, REST resources under /api/v1
Presentation	JSON serialisation; TLS encryption (HTTPS) in deployment
Session	JWT session token carried in an HttpOnly cookie across requests
Transport	TCP connections to port 5173 (GUI dev server), 8000 (API), 5432 (PostgreSQL)
Network	IPv4 loopback (127.0.0.1) in development; routable addresses in deployment

- **REST principles:** stateless requests, resource-oriented URLs (e.g. `/api/v1/prescriptions/{id}`), uniform interface.
- **CORS:** the API explicitly allows only the GUI's origin, so other websites cannot call it from a user's browser.
- **Asynchronous I/O:** Uvicorn (ASGI) handles many concurrent connections on an event loop.
- **Pagination:** list endpoints accept `page` and `per_page`, limiting payload size.
- **API documentation:** OpenAPI/Swagger is generated automatically at /docs.

10. Advanced Concepts III — Cybersecurity

Medical data is sensitive, so security follows **defence in depth** and the **principle of least privilege**.

Table 10: Security controls (aligned with OWASP Top 10)

Threat	Control implemented	Location
Password database leak	Argon2id hashing (memory-hard, unique salt per user)	security/passwords.py
Session theft via XSS	JWT stored in HttpOnly cookie, not readable by JavaScript	api/v1/auth.py
Cross-site request forgery	SameSite cookie policy + strict CORS origin list	main.py
Privilege escalation	Role-based access control via permission sets on each role class	domain/users.py, security/dependencies.py
Insecure direct object reference	Object-level checks (a patient may only read their own record)	api/v1/routes.py
Brute-force login	SlowAPI rate-limiter registered on the app (per-route limits listed as future work)	main.py
Clickjacking / MIME sniffing	X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy	main.py
Injection	ORM parameterised queries; Pydantic input validation	repositories, schemas
Secret exposure	Secrets loaded from environment (.env), not hard-coded	core/config.py

Table 11: RBAC permission matrix

Operation	Doctor	Patient	Pharmacist	Admin
Create / finalise prescription	Yes	No	No	No
View own prescriptions	Yes	Yes	—	Yes
Review prescriptions	Yes	No	Yes	Yes
Log a dose	No	Yes	No	No
Run AI interaction check	Yes	No	Yes	No
Manage users	No	No	No	Yes

11. Advanced Concepts IV — Artificial Intelligence and Analytics

11.1 Drug–Drug Interaction Checker

`LocalInteractionChecker` is a **knowledge-based (rule-based) expert system**. It holds a curated knowledge base of medicine pairs with a severity level (minor, moderate, major, contraindicated) and a clinical explanation, for example warfarin + aspirin → increased bleeding risk. Given the list of medicines on a prescription, it evaluates every pair against the rules and returns a structured `InteractionCheckResult` with the highest severity. Rule-based AI was chosen deliberately: in healthcare, results must be **explainable** and must not be invented.

11.2 Prescription OCR

`LocalOCRProvider` uses the **Tesseract** OCR engine (an LSTM neural-network recogniser) to extract text from a scanned prescription image, then parses medicine names, strengths and frequencies with pattern matching. If Tesseract is not installed, the `UnavailableOCRProvider` null object reports this honestly instead of returning fake data.

11.3 Adherence Analytics

`AnalyticsService` computes adherence as **taken doses ÷ logged doses × 100** and classifies patients as Good (≥ 80%), Needs Attention (≥ 50%) or Poor. The GUI visualises prescription volume trends and compliance with charts. This data is the natural foundation for a future machine-learning model that predicts which patients are likely to miss doses.

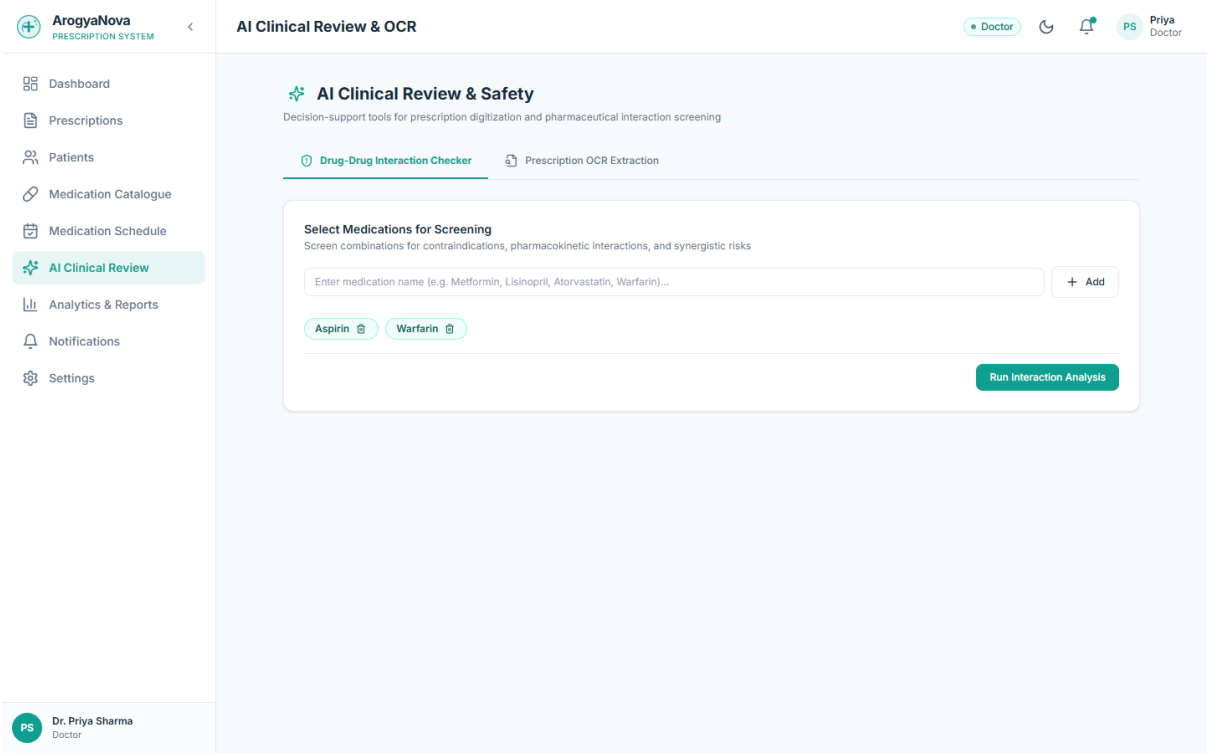


Figure 10: AI Clinical Review screen (interaction checker and OCR)

12. Graphical User Interface

The GUI is a single-page application built with **React** and **TypeScript**. It is itself object-oriented in structure: reusable components encapsulate their own markup and state, props act as each component's public interface, React contexts (`AuthContext`, `ThemeContext`) provide shared services, and the `services` folder wraps the HTTP API in typed service modules. The interface supports light and dark themes, responsive layouts and role-specific navigation.

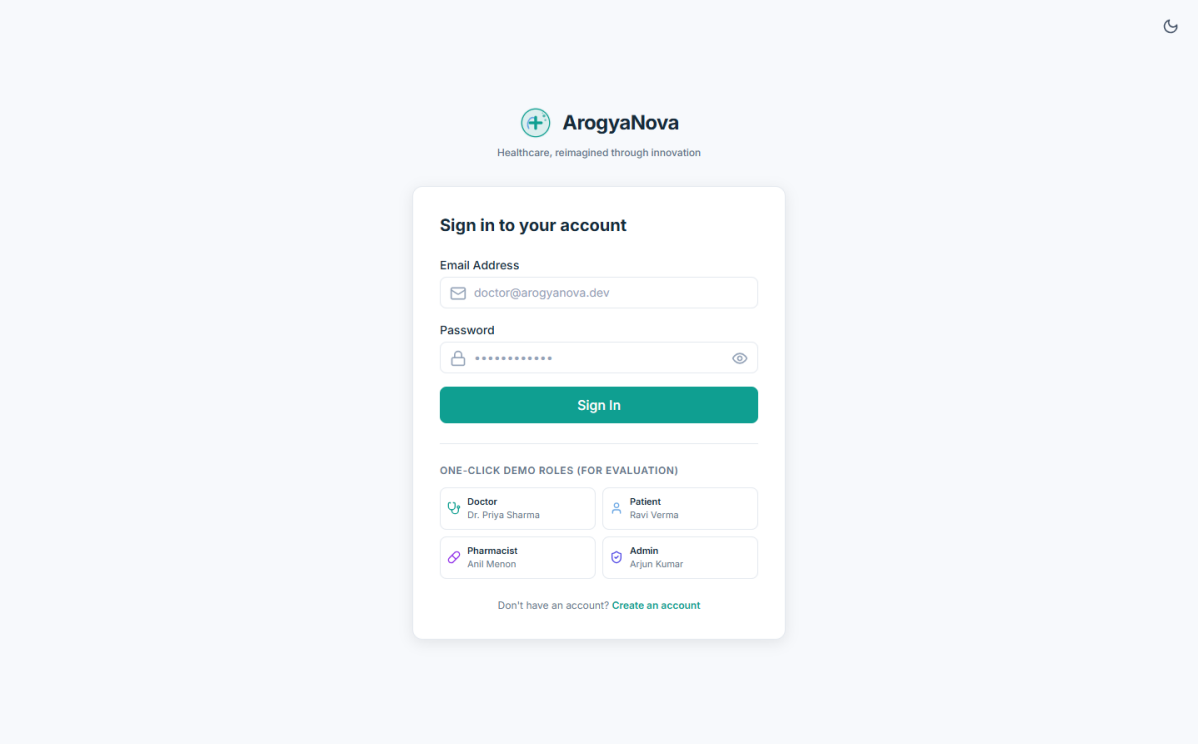


Figure 11: Login screen with one-click demo roles

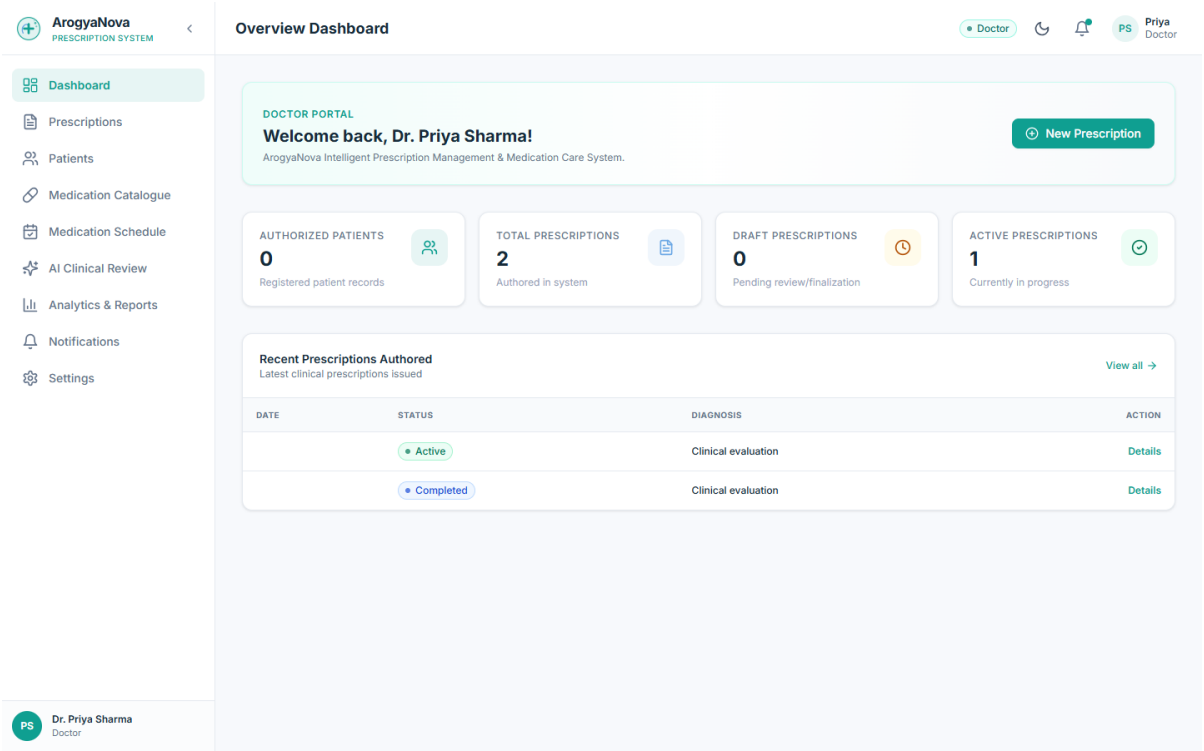


Figure 12: Doctor dashboard (light theme)

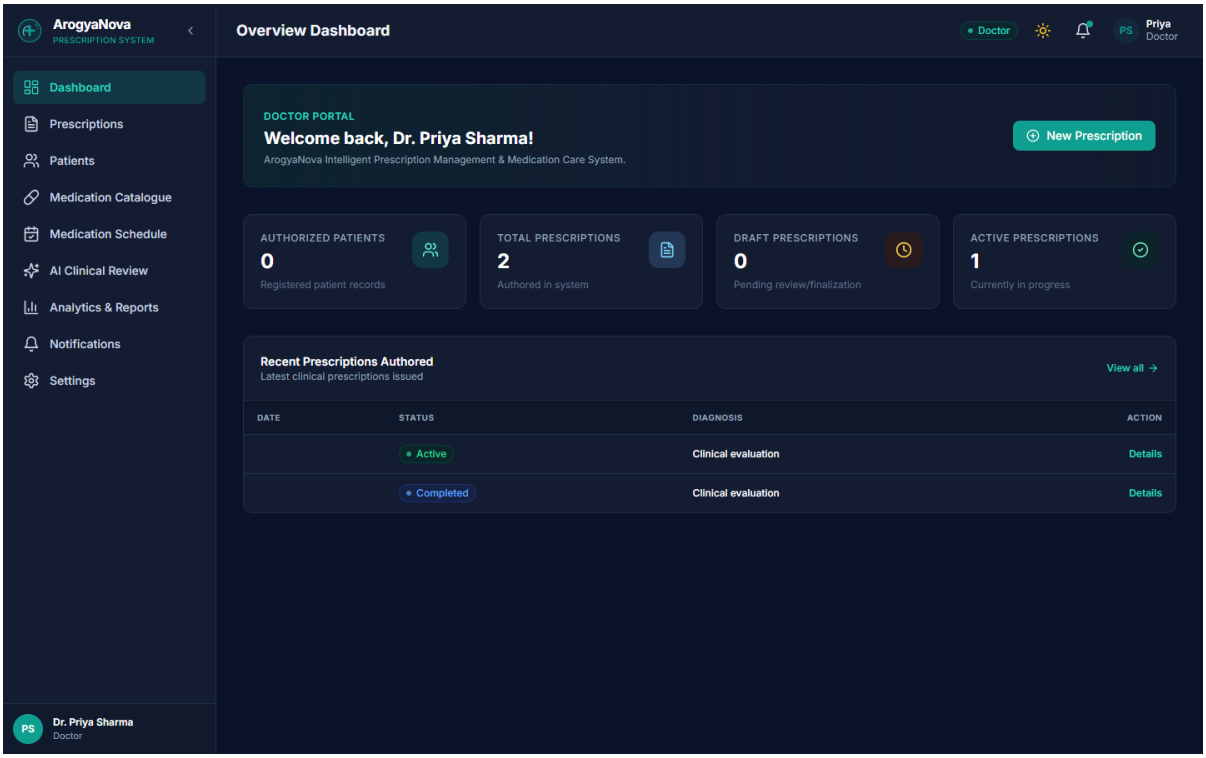


Figure 13: Doctor dashboard (dark theme)

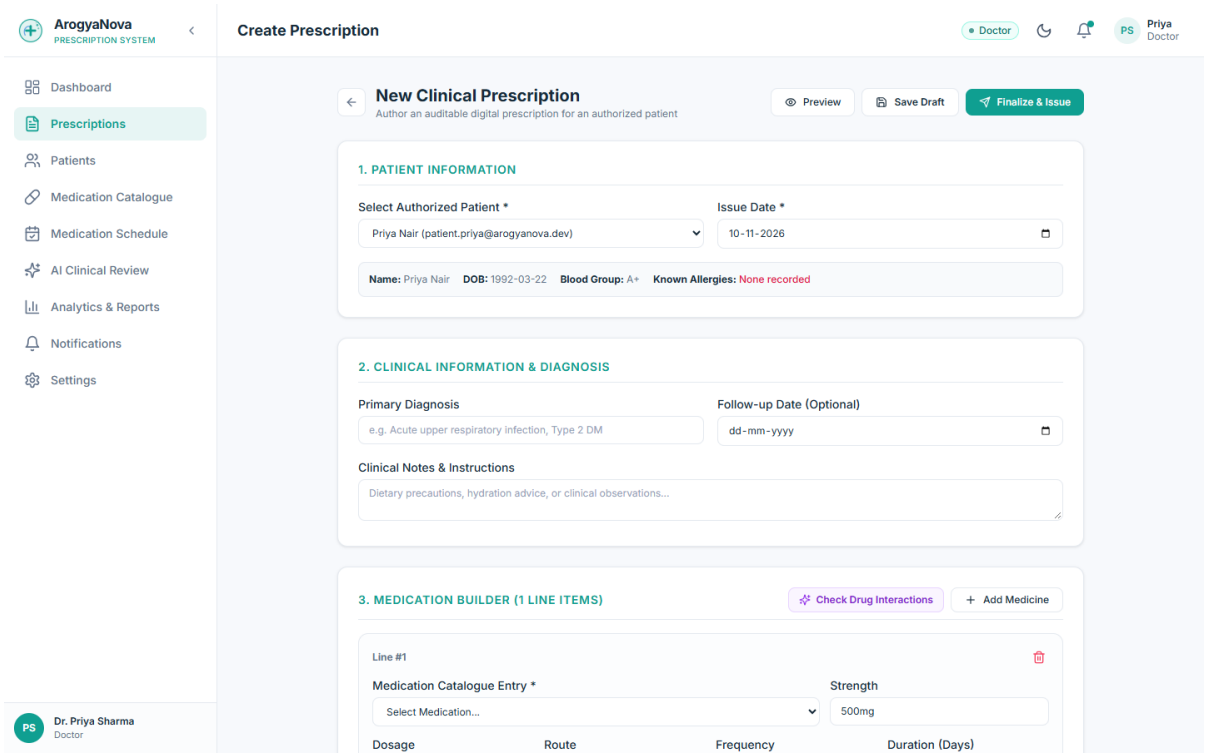


Figure 14: Create prescription form with medication builder

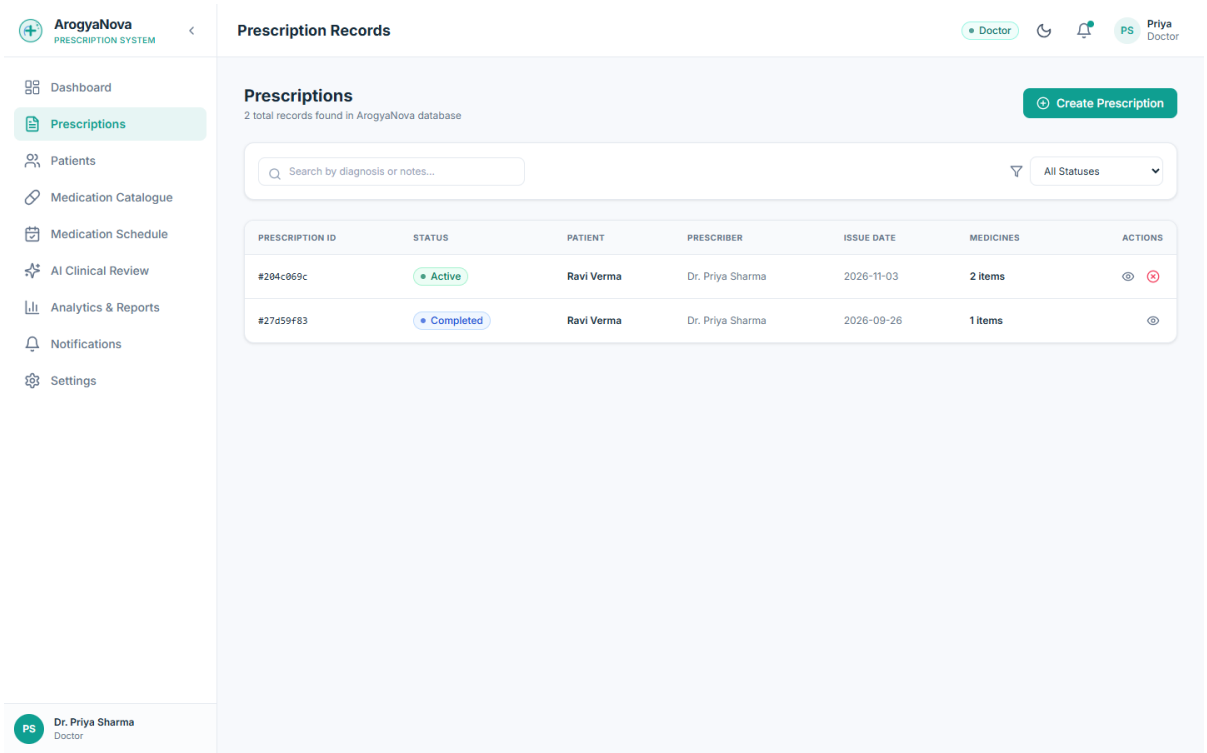


Figure 15: Prescription records list

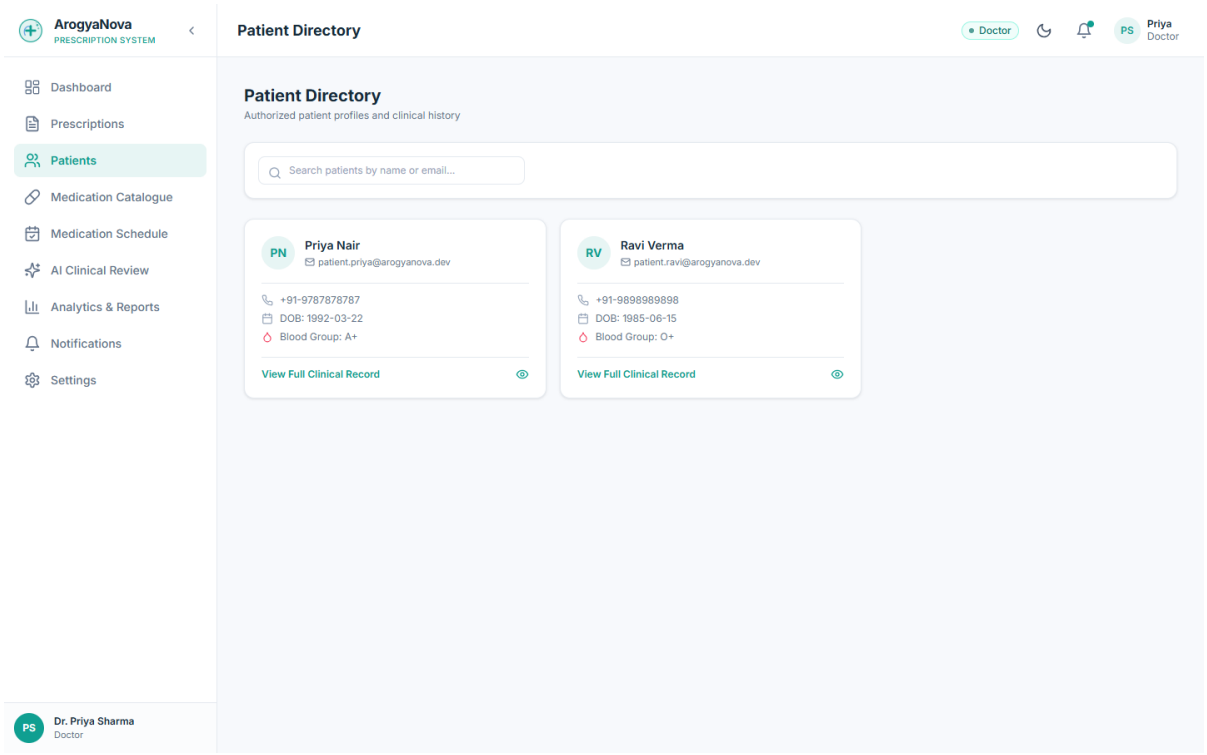


Figure 16: Patient directory

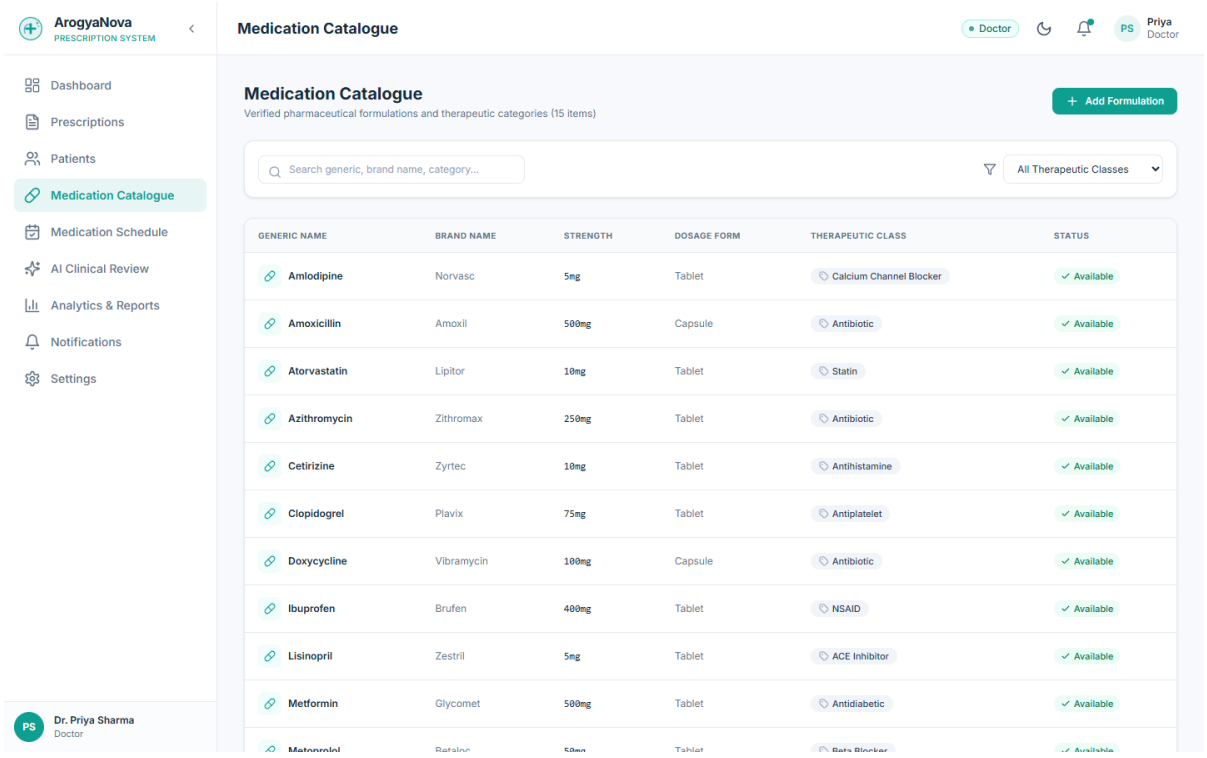


Figure 17: Medication catalogue

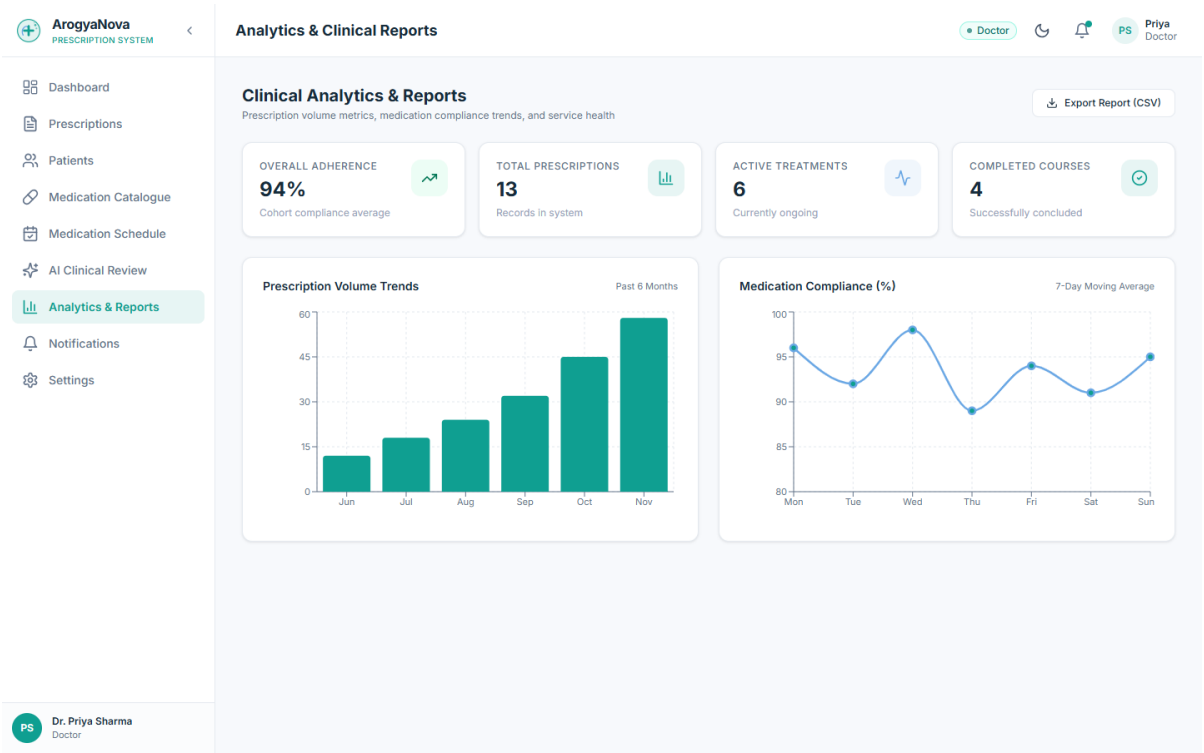


Figure 18: Analytics and clinical reports

13. Testing

The backend has an automated **pytest** suite. Because business rules live in plain domain classes, they can be tested quickly without a web server or database.

Table 12: Automated test results

Test file	Tests	What is verified
tests/test_domain.py	48	Encapsulation of Prescription, legal and illegal state transitions, role permissions, polymorphic AI and notification behaviour
tests/test_repositories.py	2	Repository CRUD contract against a database session
tests/test_security_rbac.py	4	Password hashing and role-based access rules
Total	54	All passing (pytest, 1.5 s)

```
def test_cannot_add_item_to_active_prescription(self):
    p = self._make_prescription(PrescriptionStatus.ACTIVE)
    with pytest.raises(ValueError, match="Cannot add items"):
        p.add_item(self._make_item())      # encapsulation protects the rule

def test_cannot_finalize_empty_prescription(self):
    p = self._make_prescription()
    with pytest.raises(ValueError, match="at least one medication"):
        p.finalize()
```

Manual GUI testing was carried out for each role using the seeded demo accounts. During this testing a defect was found and fixed: the patient list endpoint returned users without loading their related profile objects, which caused a server error in asynchronous mode; adding eager loading for all profile relationships resolved it.

14. Conclusion and Future Scope

14.1 Conclusion

ArogyaNova shows that object-oriented methodology is not only a modelling exercise but a practical way to make software safer. By placing clinical rules inside encapsulated domain objects, depending on abstractions, and modelling behaviour with UML before coding, the team produced a system in which illegal operations — such as editing an issued prescription — are structurally impossible. The project also demonstrates how OOM integrates naturally with databases, networking, security and AI in a real application.

14.2 Future Scope

- Train a machine-learning model on adherence logs to predict missed doses and alert doctors early.
- Apply per-route rate limits (e.g. 5 login attempts per minute) using the SlowAPI limiter already registered.
- Integrate a licensed drug-interaction database (e.g. via a standard medical API) behind the same InteractionChecker interface.
- Add WhatsApp and push-notification channels — new subclasses only, thanks to the Open/Closed design.
- Digital signatures on prescriptions using public-key cryptography.
- A mobile app reusing the same REST API.

References

- G. Booch, J. Rumbaugh, I. Jacobson — The Unified Modeling Language User Guide, 2nd ed., Addison-Wesley.
- E. Gamma, R. Helm, R. Johnson, J. Vlissides — Design Patterns: Elements of Reusable Object-Oriented Software.
- R. C. Martin — Clean Architecture: A Craftsman's Guide to Software Structure and Design.
- R. Elmasri, S. Navathe — Fundamentals of Database Systems.
- A. S. Tanenbaum — Computer Networks.
- OWASP Foundation — OWASP Top 10 Web Application Security Risks.
- FastAPI, SQLAlchemy 2.0, React and Pydantic official documentation.